



Bancos e Armazéns de Dados

*Profª Me. Claudia Maria Sodero
Salles*



Aviso legal

- Este conteúdo é parte integrante da disciplina “Bancos de Dados e Armazém de Dados”, do curso de Ciência de Dados da FATEC Baixada Santista – Rubens Lara
- Seu compartilhamento, divulgação, publicação e utilização está sujeito à autorização da professora Claudia Maria Soderro Salles, e está completamente proibido fora do âmbito da disciplina
- Santos, 2º semestre 2026

FORMAS NORMAIS



O que é a normalização



Benefícios



As formas normais



Na prática

O QUE É NORMALIZAÇÃO?

processo de modelar o banco de dados projetando a forma como as informações serão armazenadas eliminar ou minimizar a redundância no banco

O processo de normalização compreende o uso de um conjunto de regras, chamados de formas normais. Ao analisarmos o banco de dados e verificarmos que ele respeita as regras da primeira forma normal, então podemos dizer que o banco está na “primeira forma normal”. Caso o banco respeite as primeiras três regras, então ele está na “terceira forma normal”. Mesmo existindo mais conjuntos de regras para outros níveis de normalização, a terceira forma normal é considerada o nível mínimo necessário para grande parte das aplicações. [Microsoft 2007]

BENEFÍCIOS DA NORMALIZAÇÃO



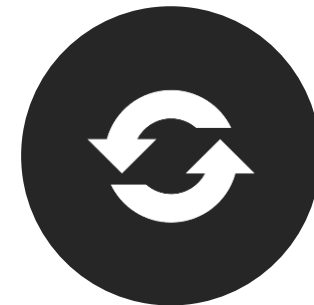
REDUÇÃO DO ESFORÇO DE
MANUTENÇÃO



OTIMIZAÇÃO DE ESPAÇO DE
ARMAZENAMENTO

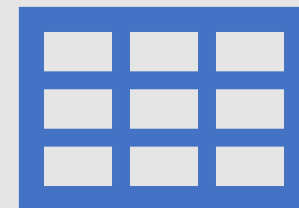


MELHORIA DA CONSISTÊNCIA E
INTEGRIDADE DOS DADOS
ARMAZENADOS



MAIOR ESTABILIDADE DO
MODELO, ADEQUA-SE COM
FACILIDADE ÀS MUDANÇAS DO
AMBIENTE

AS FORMAS NORMAIS





PRIMEIRA FORMA NORMAL

- ***PRIMEIRA FORMA
NORMAL***

Uma relação está na primeira forma normal quando todos os atributos contêm apenas um valor correspondente, singular e não existem grupos de atributos repetidos — ou seja, não admite repetições ou campos que tenham mais que um valor.

Código	Nome	Endereço	Telefone
1001	Diego Machado	Rua Tal 321 Porto	5312345678 5398765432
1002	Fulano de Tal	Avenida Tal 71 Centro	5187654321 5143215678

Tabela 1: Tabela não está na primeira forma normal

Na Tabela 1, temos 2 números de telefone para cada pessoa. Esta relação não atende à primeira forma normal, onde cada atributo contém apenas 1 valor correspondente.

Desta forma, para atender à primeira forma normal, devemos excluir a coluna (campo) telefone da tabela e criar uma nova tabela, onde cada telefone seja armazenado em uma linha (tupla ou registro), e as tabelas ligam-se pela chave (primária e estrangeira).

Código	Nome	Endereço	Bairro
1001	Diego Machado	Rua Tal 321	Porto
1002	Fulano de Tal	Avenida Tal 71	Centro

Tabela 2: Tabela está na primeira forma normal

Código	Telefone
1001	5312345678
1001	5398765432
1002	5112345678
1002	5187654321

Tabela 3: Nova tabela criada para evitar campos com mais de um valor



SEGUNDA FORMA NORMAL

- **SEGUNDA FORMA NORMAL**

Uma relação está na segunda forma normal se ela atende a todos os requisitos da primeira forma normal e se os registros na tabela, que não são chaves, dependam da chave primária **em sua totalidade** e não apenas parte dela.

A segunda forma normal trabalha com essas irregularidades e previne que haja redundância no banco de dados

cd_locacao	cd_filme	titulo_filme	devolucao	cd_cliente
1010	201	The Matrix	2011-10-12	743
1011	302	O Grito	2011-12-10	549
1012	201	The Matrix	2011-12-30	362

Tabela 4: Tabela não está na segunda forma normal

Na Tabela 4, a coluna que armazena o título do filme está associada a um código de locação, que é a chave primária. Porém, o título do filme também está associado a um código próprio, tornando-o um valor que não depende exclusivamente da chave primária desta tabela.

Desta forma, para atender à segunda forma normal, devemos excluir a coluna (campo) Título do Filme da tabela e criar uma nova tabela, onde cada Filme seja armazenado em uma linha (tupla ou registro), e as tabelas ligam-se pela chave (primária e estrangeira).

cd_filme	titulo_filme
201	The Matrix
302	O Grito

Tabela 5: Tabela criada para armazenar os filmes

cd_locacao	cd_filme	devolucao	cd_cliente
1010	201	2011-10-12	743
1011	302	2011-12-10	549
1012	201	2011-12-30	362

Tabela 6: Tabela na segunda forma normal



TERCEIRA FORMA NORMAL

- **TERCEIRA FORMA NORMAL**

Uma relação está na terceira forma normal quando não encontrarmos um atributo **não chave** dependente de outro atributo **não chave**.

Claro, contanto que esta não vá de encontro as especificações da primeira e da segunda forma normal

Para configurar uma entidade que atenda às regras da terceira forma normal, é preciso identificar as colunas (campos) que não dependem da chave primária e dependem de um outro campo não chave.

Após, é preciso separá-los para criar uma outra tabela distinta, se necessário.

Cd_func	nm_Func	Cd_depto	nm_depto
0001	Josá da Silva	01	Medicina do Trabalho
0002	João Paulo Almeida	01	Medicina do Trabalho
0003	Francisco José Pereira	03	Admissões e Rescisões
004	Hildefôncio Alves	02	Gestão de Pessoas
0005	Marcelo Simões	01	Medicina do Trabalho

Tabela 7: Tabela não está na terceira forma normal

Uma relação que contenha os atributos Matrícula do Funcionário (chave), nome do Funcionário, código do departamento e nome do departamento (Tabela 7) não atende à terceira forma normal.

O Código do Departamento e o Nome do departamento, não são atributos que pertencem ao funcionário, uma vez que o mesmo pode, eventualmente, mudar de departamento.

Para eliminar este problema, cria-se uma nova relação (Departamento), contendo o código e o nome. O Código do departamento, na tabela do funcionário, passa a ser uma chave estrangeira.

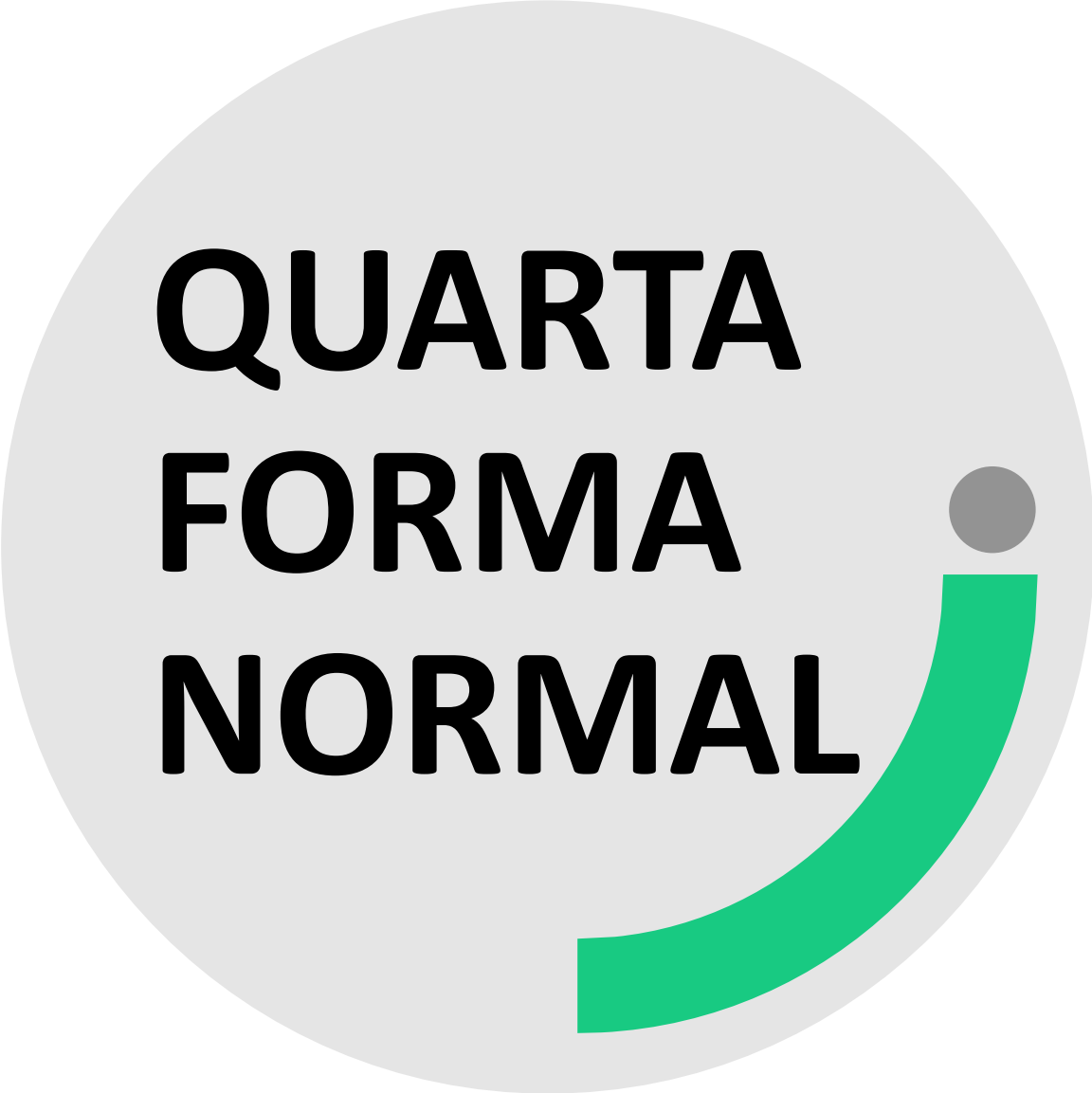
Esta forma normal também ajuda a reduzir as redundâncias de dados e melhora a integridade dos mesmos.

Cd_depto	nm_depto
01	Medicina do Trabalho
02	Gestão de Pessoas
03	Admissões e Rescisões

Tabela 8: Tabela Criada para armazenar os departamentos

Cd_func	nm_Func	Cd_depto
0001	Josá da Silva	01
0002	João Paulo Almeida	01
0003	Francisco José Pereira	03
004	Hildefôncio Alves	02
0005	Marcelo Simões	01

Tabela 9: Tabela na terceira forma normal



QUARTA FORMA NORMAL

- ***QUARTA FORMA
NORMAL***

Não devem existir dependências multivaloradas entre seus atributos

Campos que se repetem em relação a chave primária, gerando redundância nas tuplas da entidade, devem ser solucionados.

Deve-se fragmentar essa relação com o objetivo de não termos mais essas dependências funcionais do gênero.

Tipicamente, relação de muitos para muitos

musica	artista	album
Música 1	Artista 1	Álbum 1
Música 1	Artista 2	Álbum 2
Música 1	Artista 1	Álbum 2
Música 2	Artista 3	Álbum 1
Música 2	Artista 2	Álbum 1
Música 2	Artista 3	Álbum 2

Tabela 10: Tabela não está na quarta forma normal

No exemplo, uma tabela relacionando música e cantor.

Uma música pode ser interpretada por mais de um artista e este pode interpretar mais de uma música.

Para evitarmos a repetição de informações, devemos dividir a tabela.

musica	artista
Música 1	Artista 1
Música 1	Artista 2
Música 2	Artista 3
Música 2	Artista 2

Tabela 11: Tabela na quarta forma normal

musica	album
Música 1	Álbum 1
Música 1	Álbum 2
Música 2	Álbum 1
Música 2	Álbum 2

Tabela 12: Tabela na quarta forma normal



VAMOS PRATICAR?

ANALISE O MODELO AO LADO (MER) E CRIE O MODELO LÓGICO DO BANCO DE DADOS, CONSIDERANDO AS 4 FORMAS NORMAIS APRESENTADAS. A LISTA DE ATRIBUTOS DE CADA ENTIDADE E DOS RELACIONAMENTOS ESTÁ NA PRÓXIMA PÁGINA

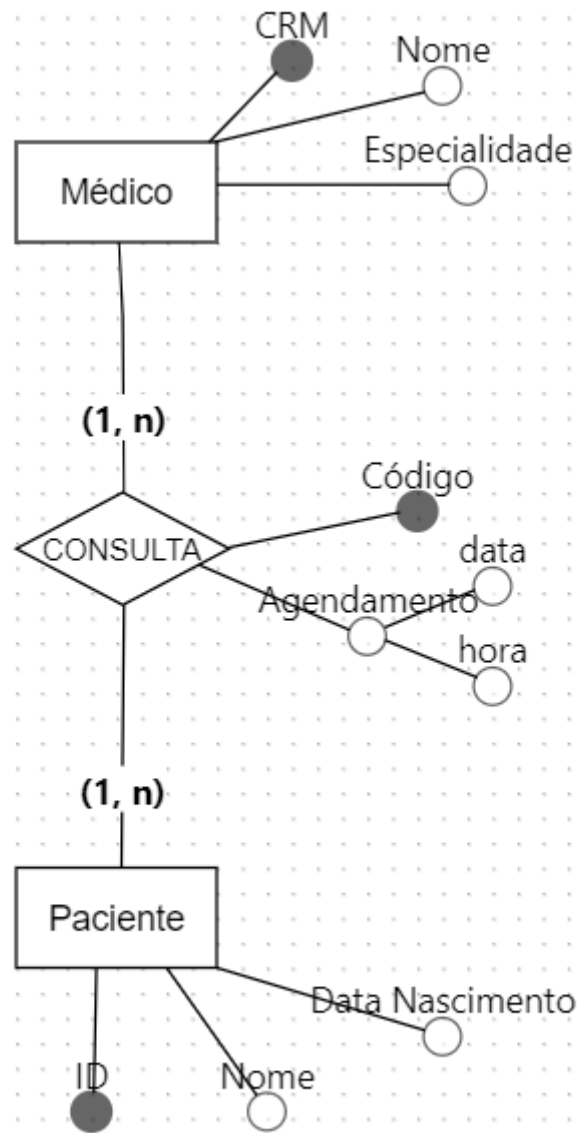
Uma clínica médica deseja informatizar o controle de suas consultas. No sistema, vários médicos atendem a vários pacientes.

Cada consulta é realizada por um médico para um paciente específico em uma data e horário específicos.

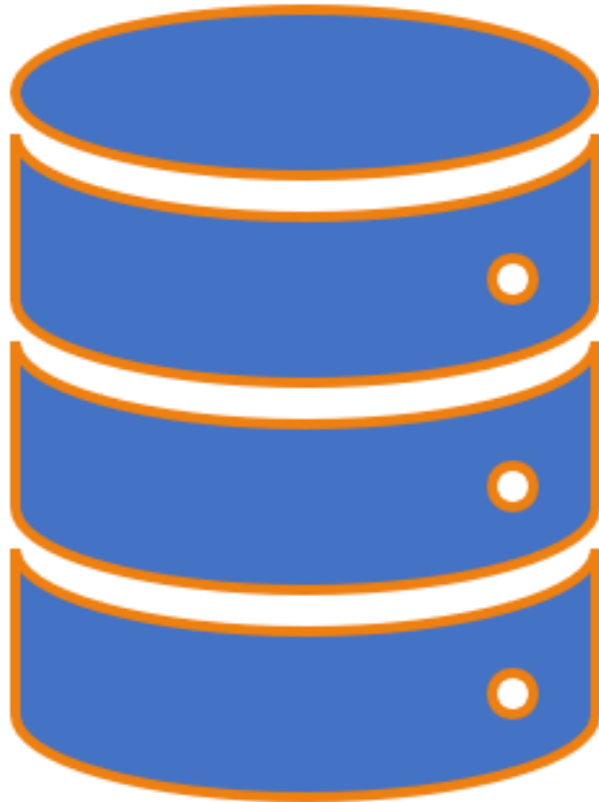
Cada médico possui especialidade, nome e CRM.

Os pacientes são identificados por um ID único, nome e data de nascimento.

As consultas são registradas com um código único, data e horário.



VAMOS FAZER A MODELAGEM LÓGICA!



O MODELO RELACIONAL

E SUAS RESTRIÇÕES

MODELO RELACIONAL

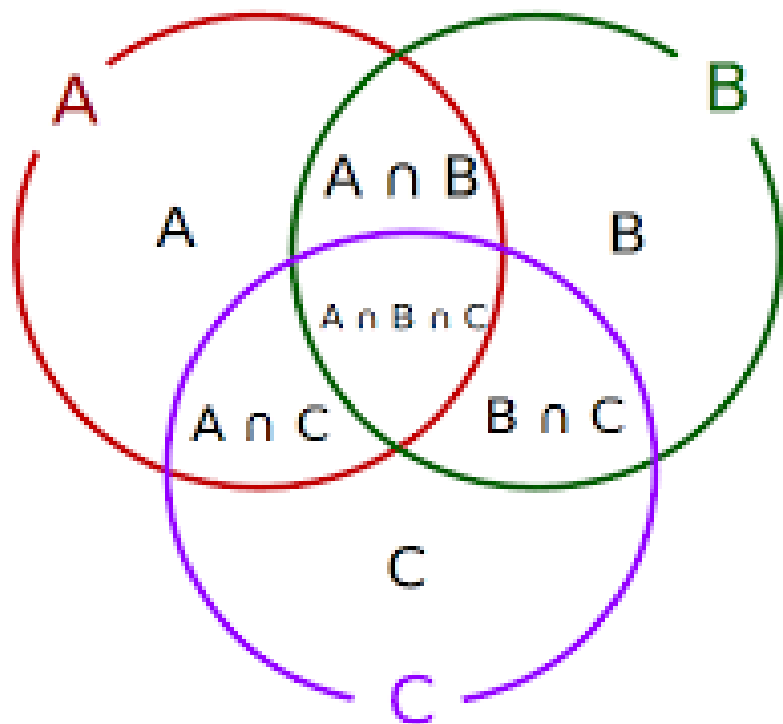
- O modelo relacional foi introduzido por Ted Codd, da IBM Research, em 1970, em um artigo clássico (Codd, 1970) que imediatamente atraiu a atenção em virtude de sua simplicidade e base matemática.
- O modelo usa o conceito de uma *relação matemática* — algo como uma tabela de valores — como seu bloco de construção básica e tem sua base teórica na teoria dos conjuntos.



TEORIA DOS CONJUNTOS

QUEM LEMBRA?

TEORIA DOS CONJUNTOS



Um conjunto é uma coleção de objetos, números ou qualquer outra coisa que possa ser agrupada, contada. Um conjunto geralmente é representado por uma letra maiúscula do alfabeto.

Todas essas partes que compõem um conjunto são chamadas de **elementos**. Podemos imaginar um cardume como um conjunto. E dentro desse conjunto, os elementos são os próprios peixes.

VOLTANDO... MODELO RELACIONAL



CONCEITOS

nome	sobrenome	Data de nascimento

- O modelo relacional representa o banco de dados como uma coleção de *relações*. Informalmente, cada relação se parece com uma tabela de valores ou, em alguma extensão, com um arquivo de registros 'plano'.

CONCEITOS

- Quando uma relação é pensada como uma **tabela** de valores, cada linha na tabela representa uma coleção de valores de dados relacionados.
- No modelo relacional, cada linha na tabela representa um fato que corresponde a uma entidade ou relacionamento do mundo real. O nome da tabela e os nomes das colunas são usados para ajudar na interpretação do significado dos valores em cada linha.

CONCEITOS

- Na terminologia do modelo relacional formal, uma linha é chamada *tupla* (ou *REGISTRO*), um cabeçalho de coluna é conhecido como *atributo*, e a tabela é chamada *relação*.
- O tipo de dado que descreve os tipos de valores que podem aparecer em cada coluna é representado pelo *domínio* de valores possíveis.

DOMÍNIO

- **Um domínio** é um conjunto de valores atômicos. Por **atômico** entendemos que cada valor no domínio é indivisível no que diz respeito ao modelo relacional. Um método comum para a especificação de um domínio é definir um tipo de dado do qual os valores de dados que formam o domínio sejam retirados. Também é útil especificar um nome para esse domínio, de modo a ajudar na interpretação de seus valores. exemplos de domínios:
 - `Numeros_fone_BR`: o conjunto de números de telefone válido no BRASIL, com ONZE dígitos.
 - `Numeros_fone_local`: o conjunto de números de telefone de NOVE dígitos, válido para um código de área em particular no BRASIL.
 - `Nomes`: o conjunto de cadeias de caracteres que representa os nomes de pessoas.
 - `Medias_pontos_graduacao`: possíveis valores de médias computadas de pontos para a graduação; cada um deve ser um número real (ponto-flutuante) entre zero e quatro.
 - `Idades_empregado`: possíveis idades dos empregados de uma empresa; cada um deve ter um valor entre 15 e 80 anos de idade.
 - `Nomes_departamento_academico`: o conjunto dos nomes dos departamentos acadêmicos em uma universidade, como ciência da computação, economia e física.

DOMÍNIO

- Um **tipo de dado** ou **formato** também é especificado para cada domínio.
 - Por exemplo, o tipo de dado para o domínio `Numeros_fone_BR` pode ser declarado como uma cadeia de caracteres no formato *(dd)dDDdd-dddd*, em que cada *d* é um dígito numérico (decimal), e os três primeiros dígitos formam um código de área de telefone válido.
 - O tipo de dado para `Idades_empregado` é um número inteiro entre 15 e 80. Para `Nomes_departamento_academico`, o tipo de dado é o conjunto de todas as cadeias de caracteres que representam nomes válidos de departamentos.
- Assim, a um domínio é dado um nome, tipo de dado e formato. Informações adicionais para interpretação dos valores de um domínio também podem ser fornecidas; por exemplo, um domínio numérico como `Pesos_pessoa` deveria ter uma unidade de medida, como libras ou quilogramas.

TIPOS DE DADOS

☰ Baseados em Caracteres:

Char(n) – Trata-se de um datatype que aceita como valor qualquer dígito, sendo que o espaço ocupado no disco é de um dígito por caractere. É possível utilizar até 8 mil dígitos.

Varchar(n) – Também aceita como valor qualquer dígito e o espaço ocupado em disco é de um dígito por caractere. Permite usar também no máximo 8 mil dígitos. A diferença pro Char, é que o Varchar geralmente é usado quando não sei o tamanho fixo de um campo.

Text – Qualquer dígito pode ser usado neste datatype, sendo ocupado 1 byte por caractere, o equivalente a **2.147.483.647** bytes.

Veja a lista completa em: <https://docs.microsoft.com/pt-br/sql/t-sql/data-types/data-types-transact-sql?view=sql-server-ver15>

TIPOS DE DADOS

- Baseados em Numéricos Inteiros:

Bigint – Aceita valores entre -2^{63} e $2^{63}-1$, sendo que esse datatype ocupa 8 bytes.

Int – Os valores aceitos aqui variam entre -2^{31} a $2^{31}-1$. Ocupa 4 bytes.

Smallint – Aceita valores entre -32768 até 32767 e ocupa 2 bytes.

Tinyint – Os valores aceitos aqui variam entre 0 e 255, ocupa apenas 1 byte.

Bit – É um tipo de dado inteiro (conhecido também como booleano), cujo valor pode corresponder a **NULL**, 0 ou 1. Podemos converter valores de string **TRUE** e **FALSE** em valores de bit, sendo que TRUE corresponde a **1** e FALSE a **0**.

Veja a lista completa em: <https://docs.microsoft.com/pt-br/sql/t-sql/data-types/data-types-transact-sql?view=sql-server-ver15>

TIPOS DE DADOS

- Baseados em Numéricos Exatos:

Decimal(P,S) – Os valores aceitos variam entre $-10^{38}-1$ e $10^{38}-1$, sendo que o espaço ocupado varia de acordo com a precisão. Se a precisão for de 1 a 9, o espaço ocupado é de 5 bytes. Se a precisão é de 10 a 19, o espaço ocupado é de 9 bytes, já se a precisão for de 20 a 28, o espaço ocupado é de 13 bytes, e se a precisão for de 29 a 38, o espaço ocupado é de 17 bytes.

Veja a lista completa em: <https://docs.microsoft.com/pt-br/sql/t-sql/data-types/data-types-transact-sql?view=sql-server-ver15>

TIPOS DE DADOS

- Baseados em Numéricos Aproximados:

Float – Este datatype aceita valores entre $-1.79E + 308$ e $1.79E + 308$. O espaço ocupado varia de acordo com o valor de n. Se esse valor estiver entre 1 e 24, a precisão será de 7 dígitos, sendo que o espaço ocupado será de 4 bytes. Se o valor de n estiver entre 25 e 53, sua precisão será de 15 dígitos, assim sendo o espaço ocupado será de 8 bytes.

Veja a lista completa em: <https://docs.microsoft.com/pt-br/sql/t-sql/data-types/data-types-transact-sql?view=sql-server-ver15>

TIPOS DE DADOS

- Baseados em Data e Hora:

Datetime – Permite o uso de valores entre **1/1/1753** e **31/12/9999**. Este datatype ocupa 8 bytes e sua precisão atinge **3.33** milisegundos.

Smalldatetime – Aceita o uso de valores entre 1/1/1900 e 06/06/2079, sendo que sua precisão é de 1 minuto e ocupa 4 bytes em disco.

Veja a lista completa em: <https://docs.microsoft.com/pt-br/sql/t-sql/data-types/data-types-transact-sql?view=sql-server-ver15>

VOLTANDO... RELAÇÃO E ATRIBUTO



RELAÇÃO E ATRIBUTO

Um **esquema de relação** é composto de um nome de relação e de uma lista de atributos

Cada **atributo A** é o nome de um papel desempenhado por algum domínio no esquema de relação.

RELAÇÃO E ATRIBUTO

- D é chamado **domínio** de A; e é indicado por **dom(A;)**.
- Um esquema de relação é usado para *descrever* uma relação; R é chamado **nome** de sua relação. O **grau** de uma relação é o número n de atributos de seu esquema de relação.
- Um exemplo :
 - ALUNO (Nome, SSN, FoneResidencia, Endereço, FoneEscritorio, Idade, MPG)
- Usando o tipo de dado de cada atributo, a definição, algumas vezes, é escrita como:
 - ALUNO (Nome: *string*, SSN: *string*, FoneResidencia: *string*, Endereço: *string*, FoneEscritorio: *string*, Idade: *integer*, MPG: *real*)
- Para esse esquema de relação, ALUNO é o nome da relação, que tem sete atributos.
- Também é possível referir-se a atributos de um esquema de relação por sua posição na relação, assim, o segundo atributo da relação ALUNO é SSN, enquanto o quarto atributo é Endereço.

Tuplas $\rightarrow$

ALUNO	Nome	SSN	FoneResidencia	Endereço	FoneEscritorio	Idade	MPG
	Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	<i>null</i>	19	3.21
	Katherine Ashly	381-62-1245	375-4409	125 Kirby Road	<i>null</i>	18	2.89
	Dick Davidson	422-11-2320	<i>null</i>	3452 Elgin Road	749-1253	25	3.53
	Charles Cooper	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
	Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	<i>null</i>	19	3.25

FIGURA 5.1 Os atributos e as tuplas de uma relação ALUNO.

CARACTERÍSTICAS DAS RELAÇÕES

- Uma relação é definida como um *conjunto* de tuplas. Matematicamente, os elementos de um subconjunto *não* têm *ordem* entre eles, portanto, as tuplas em uma relação não têm qualquer ordem em particular.
- Entretanto, em um arquivo, os registros são fisicamente armazenados em disco (ou na memória), então, sempre há uma ordem entre os registros. Essa ordenação indica o primeiro, o segundo, i-ésimos e o último registro no arquivo.
- Analogamente, quando exibimos uma relação em uma tabela, as linhas são exibidas em uma certa ordem.
- A ordenação de tupla não é parte da definição de uma relação, porque uma relação se esforça para representar fatos em um nível lógico ou abstrato. Muitas ordens lógicas podem ser especificadas em uma relação.

CARACTERÍSTICAS DAS RELAÇÕES

- Por exemplo, as tuplas da relação ALUNO poderiam ser ordenadas logicamente pelos valores de Nome, ou SSN, ou Idade, ou por algum outro atributo.
- A definição de uma relação não especifica qualquer ordem: *não* há *preferência* por uma ordenação lógica sobre outra.
- Quando uma relação é implementada como um arquivo ou exibida como uma tabela, uma ordenação em particular pode ser especificada nos registros do arquivo ou linhas da tabela.

CARACTERÍSTICAS DAS RELAÇÕES

- Valores e *Nulls* nas Tuplas.
 - Cada valor em uma tupla é um valor atômico, isto é, não é divisível em componentes dentro da estrutura do modelo relacional básico.
 - Por isso os atributos compostos e multivalorados (aulas anteriores) não são permitidos.
 - Esse modelo é chamado, às vezes, modelo relacional plano.

CARACTERÍSTICAS DAS RELAÇÕES

■ Valores e *Nulls* nas Tuplas.

- Um importante conceito se refere a ***nulls***, que são usados para representar os valores de atributos que podem ser desconhecidos ou não se aplicar a uma tupla.
- Um valor especial, chamado ***null***, é usado para esses casos. Por exemplo, algumas tuplas dos alunos têm *null* para seus fones de escritório, porque eles não têm um escritório (isto é, o fone do escritório *não se aplica* a esses alunos). Outros alunos dispõem de *null* para o fone residencial, presumivelmente porque eles não possuem um fone residencial ou têm um, mas não o conhecemos (o valor é *desconhecido*).

INTERPRETAÇÃO DE UMA RELAÇÃO

- O esquema de relação pode ser interpretado como uma declaração ou um tipo de asserção.
 - Por exemplo, o esquema da relação ALUNO declara que, em geral, uma entidade aluno tem um Nome, SSN, FoneResidencia, Endereço, FoneEscritorio, Idade e MPG.
- Cada tupla na relação pode, então, ser interpretada como um **fato** ou uma instância em particular da asserção.
 - Por exemplo, a primeira tupla estabelece o fato de que há um aluno cujo nome é Benjamin Bayer, seu SSN é 305-61-2435, sua Idade é 19, e assim por diante.

INTERPRETAÇÃO DE UMA RELAÇÃO

- Algumas relações devem representar fatos sobre *entidades*, ao passo que outras relações devem representar fatos sobre *relacionamentos*.
 - Por exemplo, o esquema de relação CURSO/HAB (SSNAluno, CodigoDepartamento) estabelece que os alunos fazem seus cursos em departamentos acadêmicos. Uma tupla nessa relação liga um aluno ao departamento de seu curso.
- Por isso o modelo relacional representa os fatos sobre ambos, entidades e relacionamentos, *indistintamente*, como relações. Às vezes, esse fato compromete o entendimento, pois é preciso supor se uma relação representa um tipo entidade ou um tipo relacionamento. (veremos mais detalhes adiante)

RESTRIÇÕES DE CHAVE

- Uma *relação* é definida como um *conjunto de tuplas*. Por definição, todos os elementos de um conjunto são distintos, por isso, todas as tuplas da relação também devem ser distintas.
- Isso significa que duas tuplas não podem ter a mesma combinação de valores para *todos* os seus atributos.

RESTRIÇÕES DE CHAVE

- Qualquer um desses conjuntos de atributos **SK** é chamado superchave do esquema de relação R.
- Uma superchave **SK** especifica uma restrição *de unicidade*, na qual duas tuplas distintas não podem ter o mesmo valor para **SK**.

RESTRIÇÕES DE CHAVE

■ Uma chave satisfaz duas restrições:

- 1. Duas tuplas distintas, em qualquer estado da relação, não podem ter valores idênticos para (todos) os atributos da chave.
- 2. Ela é uma *superchave mínima* — isto é, uma superchave da qual não podemos remover quaisquer atributos e ainda manter a restrição de unicidade garantida pela condição 1.

A primeira condição se aplica tanto às chaves como às superchaves. A segunda condição é requerida apenas para as chaves.

RESTRIÇÕES DE CHAVE

CARRO	NumeroLicenca	NumeroChassi	Marca	Modelo	Ano
	Texas ABC-739	A69352	Ford	Mustang	96
	Flórida TVP-347	B43696	Oldsmobile	Cutlass	99
	Nova York MPO-22	X83554	Oldsmobile	Delta	95
	Califórnia 432-TFY	C43742	Mercedes	190-D	93
	Califórnia RSK-629	Y82935	Toyota	Camry	98
	Texas RSK-629	U028365	Jaguar	XJS	98

A relação **CARRO** tem duas chaves candidatas: NumeroLicenca e NumeroChassi.

Em geral, um esquema de relação pode ter mais de uma chave. Nesse caso, cada uma das chaves é chamada chave candidata. É comum indicar uma das chaves candidatas como a chave primária da relação. Essa é a chave candidata cujos valores são usados para *identificar* tuplas na relação.



BANCOS DE DADOS RELACIONAIS

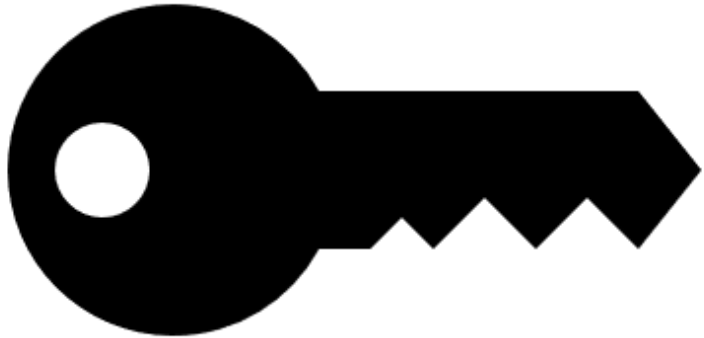
ESQUEMAS

EMPREGADO									
pnome	minicial	unome	ssn	datanasc	endereco	sexo	salario	superssn	dno
DEPARTAMENTO									
dnome	dnumero	gerssn	gerdatainicio						
DEPTO_LOCALIZACOES									
dnumero	dlocalizacao								
PROJETO									
pjnome	pnumero	plocalizacao	dnum						
TRABALHA_EM									
essn	pno	horas							
DEPENDENTE									
essn	nome_dependente	sexo	datanasc	parentesco					

Fonte: ELMASRI, 2005. pg
96

Figura 5.5 Diagrama para o esquema do banco de dados relacional EMPRESA

INTEGRIDADE DE ENTIDADE INTEGRIDADE REFERENCIAL E CHAVES ESTRANGEIRAS



INTEGRIDADE DE ENTIDADE

- A **restrição de integridade de entidade** estabelece que nenhum valor de chave primária pode ser *null*.
- Isso porque o valor da chave primária é usado para identificar as tuplas individuais em uma relação.
- Ter valores *null* para a chave primária implica não podermos identificar alguma tupla

INTEGRIDADE REFERENCIAL

As restrições de chave e as de integridade de entidade são especificadas na própria relação.

A **restrição de integridade referencial** é classificada entre duas relações e é usada para manter a consistência entre as tuplas nas duas relações.

Informalmente, a restrição de integridade referencial declara que uma tupla em uma relação, que faz referência a outra relação, deve se referir a uma *tupla existente* nessa relação.

CHAVES ESTRANGEIRAS

- Para definir integridade referencial mais formalmente definimos, primeiro, o conceito de uma *chave estrangeira*.
- As condições dadas a seguir para uma chave estrangeira especificam uma restrição de integridade referencial entre os dois esquemas de relação $R1$ e $R2$.
- Um conjunto de atributos FK do esquema da relação Rx é uma **chave estrangeira** de Rx , que faz **referência** à relação $R2$, se ele satisfizer as duas regras seguintes:
 - 1. Os atributos de FK têm o(s) mesmo(s) domínio(s) que os atributos da chave primária PK de $R2$; diz-se que os atributos de FK fazem **referência** ou se **referem** à relação $R2$.
 - 2. Um valor de FK em uma tupla t_j do estado corrente $\wedge (R \wedge$ ou ocorre como um valor de PK para alguma tupla t_2 no estado corrente $r_2(R2)$ *ou é null*. No caso anterior, temos $t_j[FK] = t_2[PK]$, e dizemos que a tupla t_x faz **referência** ou se **refere** à tupla t_2 .
- Nessa definição, $R1$ é chamada **relação referência**, e $R2$ é a **relação referida**. Se essas duas condições forem asseguradas, uma **restrição de integridade referencial** de $R1$ para $R2$ é dita garantida. Em um banco de dados com muitas relações, geralmente há muitas restrições de integridade referencial.